

Guide d'Architecture Technique et de Formation Globale - BENINCLEAN

1. Structure Globale des Repertoires de Laravel

Pour comprendre comment est structure ce projet et ou modifier les fichiers si vous voulez dupliquer le site ou changer des elements, voici une description complete des repertoires de Laravel :

- app/ : Contient le coeur de l'application en PHP. C'est ici que se trouve toute la logique metier (les modeles, les controleurs et les regles metier).
- bootstrap/ : Configure l'initialisation du framework, des middlewares de securite et de routage.
- config/ : Regroupe l'ensemble des fichiers de configuration globale (base de donnees, sessions, mail, etc.).
- database/ : Contient les scripts pour creer les tables MySQL (migrations) et generer des fausses donnees (seeders & factories).
- public/ : Le seul dossier visible par internet. Contient le point d'entree index.php, les fichiers CSS, JS et les images.
- resources/ : Contient les interfaces graphiques (les fichiers de vue Blade HTML, traduits et compiles).
- routes/ : Definit toutes les URLs de votre application web et les fonctions associees.
- storage/ : Stocke les fichiers generes en ecriture par l'application (les logs de crash, les configurations locales).
- .env : Fichier contenant les variables d'environnement. C'est ici qu'on configure le nom du site et les acces a XAMPP.

2. Guide Detaille des Fichiers du Projet (Fiche par Fiche)

Voici l'explication complete de chaque fichier cree ou modifie dans ce projet et sa fonction precise :

A. Fichier d'environnement : .env

Ce fichier sert a configurer l'application sans toucher au code PHP. Par exemple, pour changer le nom global de la plateforme, il suffit de modifier la variable APP_NAME (ex: APP_NAME=BENINCLEAN). Il sert aussi a specifier a Laravel le port et le mot de passe pour se connecter a la base de donnees MySQL de XAMPP.

B. Les Modeles de donnees (app/Models/)

Les modeles font le pont entre le code PHP et vos tables SQL via l'ORM 'Eloquent'.

- User.php : Modolise la table des utilisateurs. Contient l'attribut #[Fillable] qui securise les colonnes pouvant etre inserees par formulaire (name, email, password, role, is_active).
- Bin.php : Represente un bac a ordures avec son code unique, sa position GPS (latitude, longitude), son niveau et son statut.
- Alert.php : Lie les alertes actives ou resolues aux bacs (relation d'appartenance avec la fonction bin()).
- Collection.php : Modolise une session de ramassage de dechets planifiee ou terminee.

C. Les Controleurs (app/Http/Controllers/)

Les controleurs recoivent les requetes des utilisateurs, interrogent la base de donnees et renvoient la bonne vue HTML.

- UserController.php : Contient toute la logique de gestion des membres. La methode store() valide les entrees, crypte le mot de passe avec 'Hash::make()' et cree l'utilisateur. La methode update() modifie un utilisateur existant (permettant de changer son mot de passe). La methode toggleStatus() change l'etat actif/desactive d'un compte. La methode destroy() supprime definitivement un compte en base de donnees. Des controles de securite verifient que seul un Administrateur execute ces actions.
- BinController.php : Gere l'affichage paginee des bacs et implemente la methode updateTelemetry() qui recoit les donnees JSON du capteur IoT de l'ESP32 pour mettre a jour le niveau de remplissage, recalculer son etat et declencher/resoudre des alertes.

- SettingsController.php : Gere le formulaire de modification de la ville et des seuils. Il deplace geographiquement tous les bacs de la base de donnees autour de la nouvelle ville selectionnee et recalcule les statuts en fonction des nouveaux seuils.
- ReportController.php : Filtre et prepares les KPIs (niveaux moyens, taux d'alertes) et les series temporelles pour le graphique de niveaux de remplissage selon la periode selectionnee.
- MapController.php : Recupere la liste des bacs et calcule le point central d'affichage en fonction de la ville configuree.

D. Le Gestionnaire Persistant (app/Helpers/SettingsHelper.php)

Ce fichier a ete cree pour corriger le probleme de la deconnexion. Laravel utilise les sessions utilisateur pour stocker des donnees temporaires, mais celles-ci sont detruites lors d'une deconnexion. SettingsHelper.php contourne ce probleme en ecrivant les parametres (ville d'intervention, seuils de remplissage) dans un fichier physique 'storage/app/settings.json' sur le disque. Les parametres restent ainsi sauvegardes de maniere permanente et independante des sessions !

E. Les Vues Blade (resources/views/)

Ce sont les squelettes HTML dynamises par des balises Blade (boucles @foreach, conditions @if).

- layouts/app.blade.php : Template parent contenant le design general du site. Il integre Lucide Icons pour les icones, Leaflet pour la carte et Chart.js pour les graphiques.
- partials/sidebar.blade.php / navbar.blade.php : Les menus de navigation gauche et haut.
- dashboard.blade.php : Affiche les stats globales, la carte avec marqueurs colores (vert/jaune/rouge) et le graphique en courbes connecte a Chart.js.
- users/index.blade.php : Contient le tableau des membres avec boutons d'actions et fenetres surgissantes (modales) codees en HTML/JS pour creer et modifier les profils et les mots de passe.
- reports/index.blade.php : Affiche les rapports d'activite filtrables par dates, le bouton d'impression, et integre la feuille de style '@media print' pour masquer la navigation lors de l'impression et generer un PDF propre.

3. Guide pas-a-pas pour Concevoir un Projet Similaire

Si vous devez developper une autre application du meme type a l'avenir, suivez ces étapes :

1. Creer l'ossature : Ouvrir un terminal et lancer la commande 'composer create-project laravel/laravel nom_projet'.
2. Installer la securite : Installer Laravel Breeze avec 'composer require laravel/breeze --dev' puis la commande 'php artisan breeze:install' pour avoir les pages de Connexion, Inscription et profil pretes.
3. Creer la Base de donnees : Lancer XAMPP, ouvrir phpMyAdmin, creer une base de donnees et renseigner ses acces dans le fichier '.env'.
4. Creer les structures (Migrations) : Generer des tables SQL via Laravel en utilisant la commande 'php artisan make:migration'. Renseigner les types de colonnes (ex: string('code'), integer('fill_level'), decimal('latitude', 10, 8)). Executer les migrations avec 'php artisan migrate'.
5. Programmer la logique (Controleurs) : Creer un controleur avec 'php artisan make:controller NomController'. Y ecrire des methodes PHP pour aller chercher les donnees en base de donnees (ex: 'Bin::all()') et les envoyer aux vues.
6. Structurer les Vues : Creer des fichiers '.blade.php' dans resources/views/. Utiliser Tailwind CSS pour mettre en page des tableaux, des cartes statistiques et des boutons modernes.
7. Cartes et Graphiques : Ajouter les liens CDN de Leaflet.js et Chart.js dans votre layout de base, et utiliser Javascript pour transformer vos donnees de base de donnees (converties en JSON avec la directive '@json(\$variable)') en points sur la carte ou en graphiques dynamiques.